@ codedsoul_05

DOCKER NOTES v2.0

DOCKER

BEGINNER → ADVANCED

COMPLETE PREMIUM NOTES

From first command to production-grade containers — everything you need to learn, build and ship with confidence.

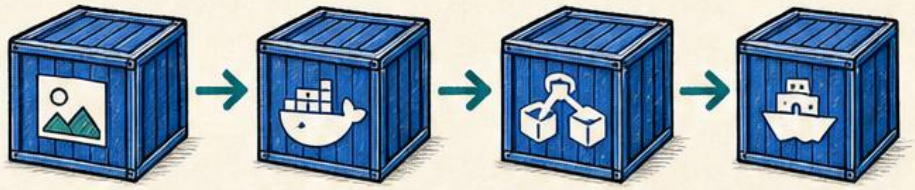


IMAGE
Blueprint of your app

CONTAINER
Isolated runtime for your app

COMPOSE
Define & run multi-container apps

SHIP
Deliver anywhere with confidence

> "Build once. Run anywhere. Ship confidently."



FAST BOOT.
Less time waiting, more time building.

CLI • Dockerfile • Volumes • Networks • Compose • Security • BuildKit

1 WHAT'S INSIDE

1. Docker Fundamentals
2. Images & Layers
3. Containers Deep Dive
4. Dockerfile Best Practices
5. Volumes & Persistence
6. Networking Concepts
7. Docker Compose
8. Security Essentials
9. Optimization & Cleanup
10. BuildKit & Caching
11. Real-World Workflows
12. Cheatsheets & Quick Recap



2 BY THE NUMBERS

- 40+ Modules
- 60+ Commands
- 15+ Cheat Sheets
- Beginner to Pro
- Security First
- Practice & Projects

3 TOPICS COVERED

- ☑ Docker Basics & CLI
- ☑ Images, Layers & Storage
- ☑ Containers & Isolation
- ☑ Dockerfile Deep Dive
- ☑ Volumes & Data Persistence
- ☑ Networking (Bridge, Host, Overlay)
- ☑ Docker Compose & Orchestration
- ☑ Secrets & Environment
- ☑ Security & Best Practices
- ☑ Logging & Monitoring
- ☑ Backup & Disaster Recovery
- ☑ Performance & Tuning
- ☑ CI/CD Integration
- ☑ Production Hardening



Build Small. Run Safe. Ship the Same Artifact.

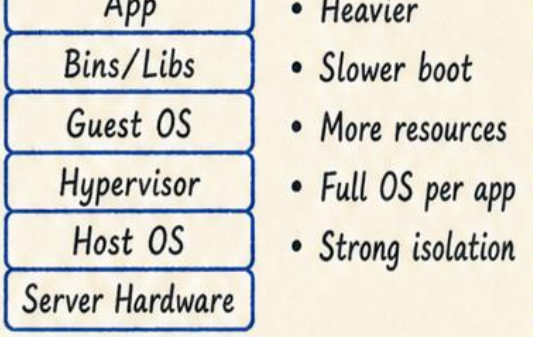
01 – WHAT & WHY DOCKER?

GOAL: Package your app + dependencies once and run anywhere consistently – dev, test, staging, prod.

THE PROBLEM: "WORKS ON MY MACHINE"

Different OS, libraries, versions, configs → unexpected bugs & delays.
Docker standardizes the environment so it just works everywhere.

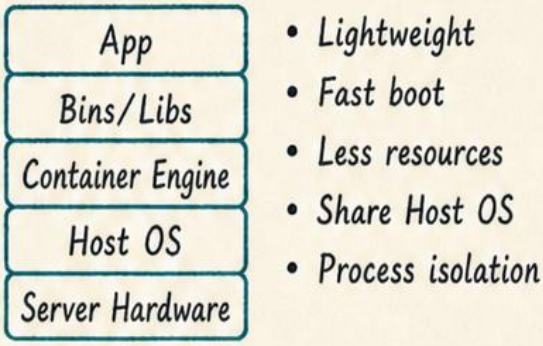
VIRTUAL MACHINE (VM)



- Heavier
- Slower boot
- More resources
- Full OS per app
- Strong isolation

Good for: Strong isolation, different OS, legacy apps.

CONTAINER (DOCKER)



- Lightweight
- Fast boot
- Less resources
- Share Host OS
- Process isolation

Good for: Microservices, CI/CD, cloud-native, rapid delivery.

NAMESPACES: Isolate processes, network, filesystem, PID, IPC, UTS, users.

CGROUPS: Limit & control CPU, memory, I/O, network usage per container.

WHERE DOCKER SHINES

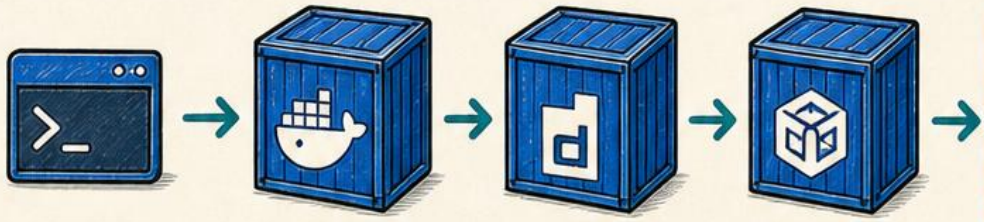
- Consistent Dev Environments
- CI/CD Pipelines
- Cloud & On-Prem Deployments
- Microservices Architecture
- Data/Tools Isolation
- Team Collaboration

TIP: Container ≠ VM

Containers share the host OS kernel → faster and smaller.
Choose containers when you need speed, density, and portability.

Same artifact from laptop to prod.

02 — ARCHITECTURE



CLI
You run commands

dockerd
Docker daemon

containerd
Manages containers

runc
Low-level runtime

CONTAINERS
Running isolated apps

IMAGES STORE
Local images on disk

REGISTRY
Remote images (Docker Hub, etc.)

CLIENT

- The CLI you interact with.
- Sends commands to the daemon.
- Supports plugins & API.

DAEMON

- Background service (dockerd).
- Builds, runs & manages containers.
- Handles images, networks, volumes.

IMAGE

- Read-only templates.
- Used to create containers.
- Stored locally or pulled.

CONTAINER

- Runnable instance of an image.
- Isolated & lightweight.
- Ephemeral by default.

NETWORK

- Connects containers.
- Built-in drivers (bridge, host, overlay, etc.).
- Enables comms.

VOLUME

- Persistent data storage.
- Outlives containers.
- Manage data separately.

NAMESPACES (ISOLATION)

- PID
- NFC
- MNT
- UTS
- USER
- CGROUP
- TIME

Each container gets its own view of the system through Linux namespaces.

CLI talks to the daemon.
Simple, powerful, containerized.

Build Small. Run Safe. Ship the Same Artifact.

03 — CLI SURVIVAL KIT

Everyday commands. Copy. Run. Ship.

PULL IMAGE

```
docker pull nginx:latest
```

Download image from Docker Hub.

LIST IMAGES

```
docker images
```

Show all local images.

RUN CONTAINER

```
docker run -d -p 8080:80 --name web nginx
```

Run in background, map port, name the container.

LIST CONTAINERS

```
docker ps
```

Show running containers.

LOGS (FOLLOW)

```
docker logs -f web
```

Stream logs from container.

EXEC SHELL

```
docker exec -it web sh
```

Open shell inside a running container.

STOP / REMOVE

```
docker stop web
docker rm web
```

Stop and remove container.

SYSTEM PRUNE

```
docker system prune -a
```

Clean up unused data.
Use with caution!

COMMAND FLOW (DAILY LOOP)



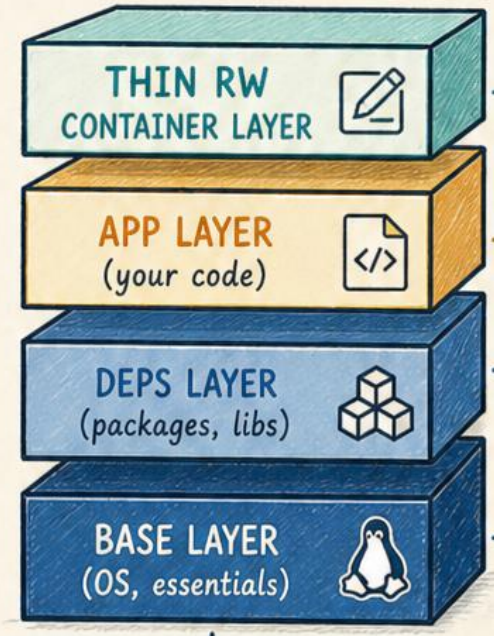
PITFALL TO AVOID

`-p host:container` → Wrong mapping = *"It works on my machine!"*

run, ps, logs, exec — daily.

04 – IMAGES & LAYERS

Images are read-only templates made of layers.
Containers add a thin writable layer on top.



Writable layer lives & dies with the container.

Your app code changes often.

Dependencies change less frequently.

Base image changes rarely.

TAGS vs DIGESTS

TAGS

human-friendly & mutable.

Example:

`nginx:1.25`

DIGESTS

immutable & content-addressed.

Example:

`nginx@sha256:a3f5c2b6...`

IMAGE HISTORY

```
docker image history <image>
```

Shows layer-by-layer build history.

IMAGE INSPECT

```
docker image inspect <image>
```

Inspect metadata, layers, digests, config & more.

OFFICIAL IMAGES

Curated by Docker.
Secure, minimal & regularly updated.

KEY TAKEAWAYS

- Layers are cached & shared across images & containers.
- Lower layers change less = faster builds & smaller pushes.
- Use specific tags or digests for reproducible deployments.

> "Understand layers today, ship faster tomorrow."

Tags are pointers. Digests are truth.

05 — DOCKERFILE BASICS

GOAL: *Build reproducible images with clear, cache-friendly Dockerfiles.*

DOCKERFILE EXAMPLE

```
1  FROM node:20-alpine          # base image
2  WORKDIR /app                 # set working dir
3  COPY package*.json ./        # copy deps first
4  RUN npm ci --omit=dev        # install prod deps
5  COPY . .                     # copy app source
6  ENV NODE_ENV=production      # environment var
7  EXPOSE 3000                  # document port
8  CMD ["node", "server.js"]    # default command
```

> *Order matters. Layer cache is your superpower.*

DOCKERFILE INSTRUCTIONS

FROM
Base image for your build.

```
FROM node:20-alpine
```

WORKDIR
Set the working directory.

```
WORKDIR /app
```

COPY
Copy files or directories.

```
COPY . .
```

RUN
Run commands in the image.

```
RUN npm ci --omit=dev
```

ENV
Set environment variables.

```
ENV NODE_ENV=production
```

EXPOSE
Document the port.

```
EXPOSE 3000
```

CMD vs ENTRYPOINT
CMD = default command (overrideable) *= fixed executable*

```
CMD ["node", "server.js"]
ENTRYPOINT ["docker-entrypoint.sh"]
```

.DOCKERIGNORE

Exclude files from build context.

```
node_modules/
.git
*.log
Dockerfile
.dockerignore
```

BUILD IMAGE

Build an image from Dockerfile.

```
docker build -t
myapp:1.0 .
```

TIP:
- *Copy dependency files first to leverage cache.*
- *Keep layers small & focused.*
- *Use .dockerignore to keep context clean.*

One concern per layer.

06 — CONTAINER LIFECYCLE

GOAL: *Understand the container lifecycle, restart behavior, signals, and data rules.*

LIFECYCLE OVERVIEW



CREATED	RUNNING	PAUSED	STOPPED	REMOVED
Container created but not started.	Container is running the main process.	Container is paused (all processes frozen).	Container is stopped but still exists.	Container is removed and cannot be started.

RESTART POLICIES

`no`	Never restart.
`on-failure`	Restart only if the container exits with a non-zero status.
`always`	Always restart unless stopped manually.
`unless-stopped`	Always restart unless previously stopped manually.

PID 1 & SIGNALS

PID 1	First process in the container.
SIGTERM	Graceful shutdown request (docker stop).
SIGKILL	Force kill after timeout.
Best Practice	Handle SIGTERM in your app for clean exits.

EPHEMERAL DATA WARNING

Container writable layer is ephemeral.
Use volumes or bind mounts to persist important data.

QUICK COMMANDS

```
docker run myapp        # Created → Running
docker pause myapp      # Running → Paused
docker unpause myapp    # Paused → Running
docker stop myapp       # Running → Stopped
docker rm myapp         # Stopped → Removed
docker restart myapp    # Restart (policy or manual)
```

PRO TIPS

- ☑ Keep containers stateless.
- ☑ Persist data in volumes or external stores.
- ☑ Use restart policies for resilience.
- ☑ Handle signals in your application.

Treat containers as cattle.

07 — VOLUMES & BIND MOUNTS

GOAL: *Persist and share data the right way.*
Choose between managed volumes, bind mounts, or tmpfs.

VOLUME

Managed by Docker. Portable & easy.

- ☑ Stored in Docker area
- ☑ Easy backup & migration
- ☑ Great for databases
- ☑ Decoupled from host path

EXAMPLE

```
# Create a volume
docker volume create data_vol
# List volumes
docker volume ls
```

USE WHEN

You want portability, backup & simplicity.

BIND

Mount host path into container.

- ☑ Direct access to host files
- ☑ Great for dev workflows
- ☑ Host path must exist
- ☑ Less portable

EXAMPLE

```
# Run with bind mount
docker run -v
$(pwd)/app:/app
myapp:latest
```

USE WHEN

You need live code, configs, or host files.

TMPFS

Memory-based filesystem.

- ☑ Stored in RAM
- ☑ Super fast
- ☑ Ephemeral (gone on stop)
- ☑ Great for sensitive data

EXAMPLE

```
docker run --tmpfs
/run --tmpfs /tmp
myapp:latest
```

USE WHEN

You need speed & don't need persistence.

TIP: Permissions & UID

Files created in a mount use container's user (often root).
Match UID/GID or use user: directive to avoid permission issues.

```
docker run -u $(id -u):$(id -g) -v $(pwd)/data:/data myapp
# or fix on host
sudo chown -R $USER:$USER ./data
```

WARNING: docker.sock ACCESS

Mounting /var/run/docker.sock gives root-equivalent access on the host.
Only do this if you fully trust the container.

```
-v /var/run/docker.sock:/var/run/docker.sock
```

Persist with volumes.

08 – NETWORKING

GOAL: Connect containers, publish ports, and choose the right network for your use case.

NETWORK DRIVERS

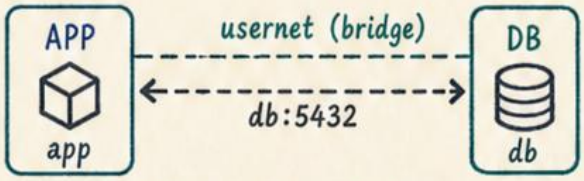
BRIDGE
Default. Containers on the same bridge can talk to each other. Outbound to the internet.

HOST
Container uses the host network stack. No network isolation. Fast, but less portable.

NONE
No networking. Use when you don't need any network access.

OVERLAY
Multi-host networking for Swarm. Connect containers across machines.

USER-DEFINED BRIDGE + DNS BY NAME



Containers resolve each other by name using built-in DNS.

EXAMPLE

```
$ docker network create usernet
$ docker run --name db --network usernet postgres
$ docker run --name app --network usernet myapp
# In app container:
# connect to database at db:5432
```

-p (PUBLISH) vs EXPOSE

-p PUBLISH
Publish container port to the host. Access from outside (host or other machines).

`-p 8080:80`

Host:8080 → Container:80

VS

EXPOSE
Document which port the container listens on. Not published to host by itself.

`EXPOSE 80`

Docs only, no host access

THE LOCALHOST TRAP

Containers don't share localhost!



Use `host.docker.internal` (Docker Desktop) or the host's IP to reach services on host.

QUICK REFERENCE

bridge	Default isolated network on one host	Most apps
host	Share host network stack	Performance tools, special cases
none	No networking	Air-gapped / offline tasks
overlay	Multi-host (Swarm)	Distributed systems

HANDY COMMANDS

```
docker network ls
docker network create usernet
docker network inspect usernet
docker run --network usernet ...
docker network rm usernet
```

User-defined bridge + DNS names.

09 – DOCKER COMPOSE

GOAL: Define & run multi-container apps with one YAML file. Consistent dev, test, and local workflows.

WHAT IS DOCKER COMPOSE?

A tool to define and run multi-container applications using a single YAML file.

Handles networking, dependencies, and service orchestration.



COMMON COMMANDS

up -d --build
Build images & start in detached mode.

down
Stop & remove containers, networks, and volumes.

logs -f [service]
Follow logs for all services or a specific service.

ps
List running services.

COMPOSE FILE (compose.yaml)

```yaml
version: "3.9"
services:
  web:
    build: .
    container_name: web
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:15-alpine
    container_name: db
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: secret
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
      interval: 5s
      timeout: 5s
      retries: 10
volumes:
  db_data:
```

DEPENDENCIES & HEALTH

- depends_on with condition: service_healthy waits until the dependency is healthy.
- Use healthcheck in services to define readiness.
- Better startup order, fewer race conditions.

BEST PRACTICES

- ☑ Keep compose.yaml in project root.
- ☑ Use env_file for secrets & config.
- ☑ Pin image versions for stability.
- ☑ Use healthchecks for critical deps.
- ☑ Persist data with named volumes.
- ☑ One process per container.

Multi-container apps = Compose first.

10 — MULTI-STAGE BUILDS

GOAL: *Use multi-stage builds to keep images small, secure, and production-ready.*

WHY IT MATTERS:

Build tools & deps aren't needed in production.
Multi-stage = smaller attack surface & faster deployments.

BUILDER STAGE



- *Full toolchain*
- *Build app*
- *Compile assets*
- *Install deps*

COPY --from=builder

```
COPY --from=builder
/app/bin/myapp /app/
```

RUNTIME STAGE



- *Minimal base image*
- *Only runtime deps*
- *App binary/assets*
- *Non-root user*

BENEFITS

SMALLER IMAGES
Ship only what's needed to run.

BETTER SECURITY
Fewer packages, fewer risks.

FASTER DEPLOYS
Less to pull, start & update.

LEAST PRIVILEGE
Run as non-root in final stage.

PRO TIP: ORDER FOR LAYER CACHE

1. *Base Image* →
2. *Copy Manifests (package.json, go.mod)* →
3. *Install Deps* →
4. *Copy Source* →
5. *Build*

Changes to source won't bust earlier cached layers.

> *Build clean.*
> *Ship lean.*
> *Run secure.*

```
FROM golang:1.22 AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN go build -o /out/myapp ./cmd/app
```

```
FROM gcr.io/distroless/base-debian12
WORKDIR /app
COPY --from=builder /out/myapp /app/
USER nonroot:nonroot
ENTRYPOINT ["/app/myapp"]
```

Final stage = only what you need to run.

11 — REGISTRY, TAGS & DIGESTS

GOAL: Version, share, and pull images securely and reliably.

THE FLOW: BUILD → TAG → LOGIN → PUSH → PULL



- `docker build -t myapp:dev .`
- `docker tag myapp:dev registry.io/myapp:1.0.0`
- `docker login registry.io`
- `docker push registry.io/myapp:1.0.0`
- `docker pull registry.io/myapp:1.0.0`

TAGS: HUMAN-FRIENDLY ALIASES

- latest, 1.0.0, v1, dev, prod...
- Mutable → can be moved/overwritten
- Use semantic versions for releases
- Use git-sha tags for traceability

EXAMPLES:

`myapp:latest`	Point to newest
`myapp:1.0.0`	Release version
`myapp:git-sha`	Immutable build ref

DIGESTS: IMMUTABLE & PRECISE

- Content addressable (by image layers)
- Never changes → safe for prod
- Format: repo@sha256:<hash>

EXAMPLE:

`myapp@sha256:5e9c...3f7a`

Best practice: pin by digest in production.

PRIVATE REGISTRIES

- Docker Hub (public/private)
- Amazon ECR, Google GAR, Azure ACR
- Self-hosted: Harbor, Nexus, GitLab
- Use auth + RBAC for access control

INSPECT & FIND DIGESTS

- docker inspect
- docker image inspect
- docker buildx imagetools inspect

GET DIGEST:

```
docker buildx imagetools inspect myapp:1.0.0
# Look for "Digest: sha256:..."
```

BUILD & PUSH (EXAMPLE)

```
$ docker build -t myapp:git-sha .
$ docker tag myapp:git-sha \
    registry.io/myapp:git-sha
$ docker push registry.io/myapp:git-sha
# Also tag a release
$ docker tag myapp:git-sha \
    registry.io/myapp:1.0.0
$ docker push registry.io/myapp:1.0.0
```

PULL BY DIGEST (PROD)

```
$ docker pull myapp@sha256:5e9c...3f7a
$ docker run -d --name myapp \
    myapp@sha256:5e9c...3f7a

# Or in Compose
  image: myapp@sha256:5e9c...3f7a
```

REFERENCE COMMANDS

`docker images`	List local images & tags
`docker login registry.io`	Login to registry
`docker logout registry.io`	Logout
`docker image ls --digests`	Show digests
`docker image inspect myapp:1.0.0`	Inspect image
`docker buildx imagetools inspect myapp:1.0.0`	Inspect tags & digest
`docker pull myapp:tag`	Pull by tag
`docker pull myapp@sha256:...`	Pull by digest

Pin digests in production.

12 – RESOURCES, HEALTH & LOGS

GOAL: *Keep containers healthy, use resources wisely, and make logs useful.*

1. RESOURCES

Limit what containers can use so your host stays stable.

```
docker run \
  --memory=512m \
  --cpus="1.5" \
  my-app
```

- `--memory`: *RAM limit*
- `--cpus`: *CPU limit (e.g., 1.5 = 1.5 cores)*

Right-size for performance & cost.

2. HEALTHCHECK

Tell Docker how to check if your container is healthy.

```
HEALTHCHECK --interval=30s \
      --timeout=5s \
      --retries=3 \
 CMD curl -f http://
  localhost:8080/health
  || exit 1
```

- `interval`: *how often*
- `timeout`: *max time to check*
- `retries`: *failures before unhealthy*

Docker marks container healthy / unhealthy.

3. COMPOSE HEALTH & DEPENDS_ON

Make services start in the right order and stay aware of health.

```
services:
  web:
    image: my-web
    healthcheck:
      test: ["CMD", "curl", "-f",
       "http://localhost:8080/health"]
      interval: 30s
      timeout: 5s
      retries: 3
  api:
    image: my-api
    depends_on:
      web:
        condition: service_healthy
```

API waits until Web is healthy.

4. LOGS & ROTATION

Keep logs useful and avoid filling the disk.

```
# daemon.json
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

- `max-size`: *per-file size*
- `max-file`: *how many files to keep*

Keeps last (max-file) files, then rotates.

5. DOCKER STATS

See live usage of containers.

```
$ docker stats
CONTAINER      CPU %      MEM USAGE      MEM %
web            12.34      120MiB         15.2%
api            20.11      180MiB         22.7%
worker          5.02       80MiB          9.1%
```

- `CPU %`: *current CPU usage*
- `MEM USAGE`: *memory in use*
- `MEM %`: *memory vs limit*

TIP: *Set sensible limits, define healthchecks, and rotate logs. Your containers (and future you) will thank you.*

Limit. Healthcheck. Rotate logs.

13 – SECURITY HARDENING

GOAL: Minimize risk & blast radius – secure images, runtime, and supply chain.

1 RUN AS NON-ROOT USER



Don't run as root. Create a dedicated user and set USER.

2 USE SLIM OR DISTROLESS



Use minimal base images. Fewer packages, smaller attack surface.

3 NO SECRETS IN LAYERS



Never bake secrets into images or ENV. They persist in layers.

4 USE BUILD SECRETS



Use Docker Build Secrets (--secret) for tokens, keys, and credentials.

5 DROP CAPABILITIES



Drop all capabilities and add only what's required.

6 NO PRIVILEGED CONTAINERS



Avoid --privileged. It breaks isolation and increases risk.

7 SCAN YOUR IMAGES



Continuously scan for vulnerabilities (OS & dependencies).

8 PIN IMAGE DIGESTS



Use image digests instead of tags for repeatable, trusted builds.

9 AVOID DOCKER.SOCK



Don't mount the Docker socket unless absolutely necessary.

10 ROOTLESS WHERE POSSIBLE



Use rootless Docker when supported. Safer by default.

TIP: Combine secure images + least-privilege runtime + strong supply chain for defense in depth.

KEY
- ☑ Do this
- Reduces risk
- Detect & verify

Least privilege. Scan. Never bake secrets.

14 — DEBUGGING & TROUBLESHOOTING

GOAL: Find issues fast, understand what broke, and fix with confidence.

WHAT BREAKS → HOW TO FIX

WHAT BREAKS	HOW TO FIX
App errors, crashes, or unexpected behavior →	**TOOLS:** `docker logs [-f] <container>` **FIX:** Check logs, look for stack traces & error messages.
Not sure about config, env, mounts, ports, or network →	**TOOLS:** `docker inspect <container|image>` **FIX:** Inspect JSON, verify settings, env, mounts, networks.
Need to run commands inside a running container →	**TOOLS:** `docker exec -it <container> sh` **FIX:** Shell in and debug live. Check files, env, processes.
Not sure what changed or who did what →	**TOOLS:** `docker history <image>` **FIX:** Review layers, sizes, commands, and authors.
Builds fail or unclear why a layer broke →	**TOOLS:** `docker build --progress=plain .` **FIX:** See full build output, pin versions, clean cache.

DEBUGGING TOOLKIT

`docker logs [-f]`	`docker inspect`	`docker exec -it`	`docker history`	`docker build --progress=plain`

TIP: Reproduce → Isolate → Fix

Reproduce the issue in a minimal setup. Isolate the cause. Apply the smallest fix. Rebuild. Retest. Repeat.

Logs → inspect → exec.

15 — BUILDKIT, BUILDX & CACHE

GOAL: Build faster, smarter, and safer.
Smaller images, secure builds, blazing CI.

THE PROBLEM:
Naive builds are slow and leaky.
Dependencies re-download, secrets leak, CI takes forever.

BUILDKIT BENEFITS

- ☑ Parallel build steps
- ☑ Smarter caching
- ☑ Smaller contexts
- ☑ Secrets management
- ☑ Better performance

Enable it:

```
DOCKER_BUILDKIT=1
```

CACHE MOUNTS

Reuse package manager cache across builds.

```
RUN --mount=type=cache,
  target=/root/.cache \
  pip install -r
  requirements.txt
```

Faster installs, smarter layers.

SECRET MOUNTS

Use secrets without baking into layers.

```
RUN --mount=type=secret,
  id=mytoken \
  export TOKEN=$(cat
  /run/secrets/mytoken) \
  && make deploy
```

Secrets stay out of layers and history.

BUILDX MULTI-ARCH

Build & push multi-arch images easily.

```
docker buildx create --use
docker buildx build \
  --platform linux/amd64,
    linux/arm64 \
  --t myapp:latest --push
```

*One command.
Multiple architectures.*

REGISTRY CACHE (CI)

Cache layers in registry to speed up CI.

```
docker buildx build \
  --cache-from=type=registry,
  ref=myrepo/myapp:buildcache \
  --cache-to=type=registry,
  ref=myrepo/myapp:buildcache,
  mode=max \
  -t myapp:latest --push
```

*Share cache.
Build faster.*

.DOCKERIGNORE

*Keep context small.
Build what matters.*

```
node_modules
.git
logs
*.log
.env
Dockerfile*
README.md
```

Smaller context, faster builds.

BEST PRACTICE: Use BuildKit features + buildx + registry cache for fast, reproducible, and secure builds.

★ Cache mounts speed CI. Secrets stay out of layers. ★

16 — COMPOSE FOR REAL APPS

GOAL: Use advanced Compose features to run reliable, real-world apps in dev and small prod.

OVERRIDE FILES

- Layer Compose files for different environments.
- Base + override = final config.

```
docker compose -f compose.yml \
    -f compose.override.yml \
    -f compose.prod.yml up -d
```

PROFILES

- Enable optional services without maintaining multiple files.

```
services:
  smtp:
    image: mailhog/mailhog
    profiles: ["dev", "test"]
```

Run with: `--profile dev`

ENV_FILE

- Load environment variables from files.

```
services:
  web:
    env_file:
      - .env
      - .env.local
```

Tip: later files override earlier ones.

RESTART POLICY

- Control container restart behavior.

```
services:
  web:
    image: myapp/web
    restart: unless-stopped
```

Options: no | always | on-failure | unless-stopped

HEALTHCHECK + DEPENDS_ON

- Ensure services are healthy before starting dependents.

```
services:
  api:
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/health"]
      interval: 10s
      retries: 5
  web:
    depends_on:
      api:
        condition: service_healthy
```

INIT (TINI) FOR CLEAN SIGNALS

- Use a minimal init to handle PID 1 and reap zombie processes.

```
services:
  worker:
    image: myapp/worker
    init: true
    command: ["python", "worker.py"]
```

Equivalent to: `docker run --init`

STOP GRACE PERIOD

- Give containers time to shut down nicely.

```
services:
  web:
    stop_grace_period: 30s
```

Sends SIGTERM, then SIGKILL after timeout.

WHEN TO MOVE TO KUBERNETES

- Complex scheduling, autoscaling, multi-region, or strict HA requirements.
- Heavy traffic, multi-tenancy, or advanced observability needs.

BEST PRACTICES

- ☑ Keep compose.yml minimal & portable.
- ☑ Use override files per environment.
- ☑ Use healthchecks for critical services.
- ☑ Externalize secrets (don't commit).
- ☑ Use profiles for dev/test tooling.
- ☑ Set restart policies intentionally.
- ☑ Use .env files, not hardcoded values.
- ☑ Use init for long-running processes.
- ☑ Know limits; scale wisely.

Compose for dev & small prod. Know the limits.

17 — BEYOND ONE HOST

From single machine to clusters.
Two popular paths:

DOCKER SWARM VS **KUBERNETES**

CONCEPT MAP

CONTAINER	≈ Same idea	POD CONTAINER
COMPOSE SERVICE	≈ Similar abstraction	DEPLOYMENT
VOLUME	≈ Persistent storage	PERSISTENT VOLUME (PV)
NETWORK	≈ Service discovery & routing	SERVICE
STACK (SWARM)	≈ Collection of services & infra	NAMESPACE / HELM RELEASE

WHAT ORCHESTRATORS ADD

SCHEDULER
Places containers where resources are available.

DESIRED STATE
You declare the desired state, the system makes it real.

SELF-HEALING
Detects issues and restarts or replaces failed containers.

ROLLING UPDATES
Update without downtime. Roll forward or roll back.

SCALING
Scale up/down based on load or rules.

YOU (DECLARE) → ORCHESTRATOR (SCHEDULER + CONTROL PLANE) → CLUSTER (NODES)

Reconcile → Act → Observe → Repeat

TIP: Learn Docker & fundamentals first. Master containers, images, networks, volumes. Then Kubernetes will click!

Containers first. Orchestrate when you outgrow one box.

18 — COMMAND CHEAT SHEET

Your daily dockertools. Copy. Run. Ship.

IMAGES

`docker images`	List images
`docker pull <image>`	Pull image
`docker rmi <image>`	Remove image
`docker image prune`	Remove unused images
`docker history <image>`	Image history
`docker inspect <image>`	Inspect image
`docker tag <src> <dst>`	Tag image

VOLUMES

`docker volume ls`	List volumes
`docker volume create <name>`	Create volume
`docker volume inspect <name>`	Inspect volume
`docker volume rm <name>`	Remove volume
`docker volume prune`	Remove unused volumes

NETWORKS

`docker network ls`	List networks
`docker network create <name>`	Create network
`docker network inspect <name>`	Inspect network
`docker network rm <name>`	Remove network
`docker network prune`	Remove unused networks
`docker network connect <net> <c>`	Connect container
`docker network disconnect <net> <c>`	Disconnect container

BUILD

`docker build -t <image> .`	Build image
`docker build -f <file> -t <img> .`	Build from Dockerfile
`docker build --no-cache .`	Build no cache
`docker build --pull .`	Build with latest base
`docker buildx build -t <img> .`	Build with Buildx
`docker buildx bake`	Build from bake file

CONTAINERS

`docker ps`	List running containers
`docker ps -a`	List all containers
`docker run <image>`	Run a container
`docker run -it <image> sh`	Run interactively
`docker stop <id|name>`	Stop container
`docker start <id|name>`	Start container
`docker restart <id|name>`	Restart container
`docker rm <id|name>`	Remove container
`docker rm -f <id|name>`	Force remove
`docker logs -f <id|name>`	Follow logs
`docker exec -it <id> sh`	Shell into container
`docker inspect <id|name>`	Inspect container

COMPOSE

`docker compose up`	Up services
`docker compose up -d`	Up detached
`docker compose down`	Stop & remove
`docker compose ps`	List services
`docker compose logs -f`	Follow logs
`docker compose exec <svc> sh`	Exec in service
`docker compose build`	Build services
`docker compose pull`	Pull services
`docker compose restart`	Restart services
`docker compose config`	Validate config

SYSTEM

`docker info`	System information
`docker version`	Version details
`docker system df`	Disk usage
`docker system prune`	Prune unused data
`docker system prune -a`	Prune all unused
`docker events`	Stream events

PRO TIPS

- Add `-a` to list everything.
- Use `-f` with rm/stop to force.
- Logs are your best friend: `logs -f`
- Compose + Volumes = Happy Data.

★ Muscle-memory the top 15. ★

19 – ROADMAP & MASTER RECAP

LEARNING PATH (WEEK-BY-WEEK)

WEEK 1	WEEK 2	WEEK 3	WEEK 4	WEEK 5	WEEK 6	WEEK 7+
Docker Fundamentals	Images, Containers & Layers	Dockerfile Best Practices	Volumes, Networks & Persistence	Docker Compose	Security, Cleanup & Optimization	BuildKit, CI/CD & Production

TOP TALKING POINTS (HIGH IMPACT)

- ☑ Containers package app + dependencies
- ☑ Same artifact across dev → prod
- ☑ Images are immutable, containers are runtime
- ☑ Layers = speed via caching
- ☑ Volumes for persistence, not in-container data
- ☑ Networks enable secure communication
- ☑ Security first: least privilege, small images
- ☑ Automate with Compose, CI/CD, and BuildKit

MINI Q&A (INTERVIEW + REAL WORLD)

CMD vs ENTRYPOINT?

CMD	Default command, can be overridden.
ENTRYPOINT	Fixed command, not easily overridden.
Rule	Use ENTRYPOINT for main process.

ADD vs COPY?

COPY	Copy files & folders. Simple & safe.
ADD	Also extracts archives, handles URLs.
Rule	Use COPY unless you need ADD features.

User-defined networks?

Why	Better isolation, DNS, and control.
Benefit	Containers talk by name, not IP.
Use	Always over default bridge network.

Where to store secrets?

Don't	Never in Dockerfile or image layers.
Do	Use Docker Secrets / env / vaults.
Why	Keep secrets out of images & history.

COMMON MISTAKES (AVOID THESE!)

- ✗ Running containers as root
- ✗ Using latest tag in production
- ✗ Not using .dockerignore
- ✗ Storing secrets in images
- ✗ Not cleaning up unused resources
- ✗ Overusing ADD when COPY is enough
- ✗ Binding to 0.0.0.0 without firewall
- ✗ Ignoring image size & layers
- ✗ No resource limits on containers
- ✗ Not automating builds & deployments

BUILD SMALL. RUN SAFE. SHIP THE SAME ARTIFACT.